INTERNSHIP REPORT

A report submitted in partial fulfilment of the requirements for the Award of Degree of

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING

by

P DEVANAND

Regd.No:21781A04E8

Under Supervision of

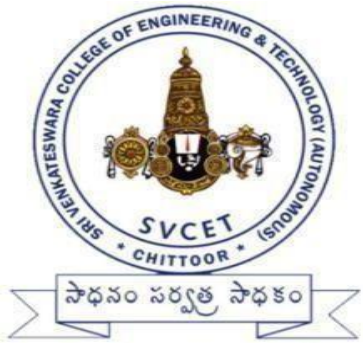
VISHNU P NAIR

INTRAINZ INNOVATIONS PVT. LTD

(Duration: 01/02/2025 to 15/03/2025)

SRI VENKATESWARA COLLEGE OF ENGINEERING& TECHNOLOGY
(AUTONOMOUS)
R.V.S NAGAR, CHITTOOR – 517 127. (A.P)
(Approved by AICTE, New Delhi, Affiliated to JNTUA, Anantapur)
(Accredited by NBA, New Delhi & NAAC, Bengaluru)
(An ISO 9001:2000 Certified Institution)
2024-2025

SRI VENKATESWARA COLLEGE OF ENGINEERING& TECHNOLOGY
(AUTONOMOUS)
R.V.S NAGAR, CHITTOOR – 517 127. (A.P)
(Approved by AICTE, New Delhi, Affiliated to JNTUA, Anantapuramu)
(Accredited by NBA, New Delhi & NAAC, Bengaluru)
(An ISO 9001:2000 Certified Institution)



CERTIFICATE

This is to certify that the "**Internship report"** submitted by **PERUGU DEVANAND (Regd.No:21781A04E8)**is work done by him and submitted during 2024- 2025 academic year, in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY in ELECTRONICS AND COMMUNICATION ENGINEERING,** at INTRAINZ

Dr.J.SATHEESH KUMAR
Internship Coordinator

Dr.D.SRIHARI
Head of the Department

Review Conducted on:_____

INTERNAL EXAMINER **EXTERNAL EXAMINAR**

CERTIFICATE OF INTENSHIP

ACKNOWLEDGEMENT

A grateful thanks to **Dr.R.Venkataswamy, Chairman** of Sri Venkateswara College of Engineering & Technology for providing education in their esteemed institution.

I wish to record my deep sense of gratitude and profound thanks to our beloved **Vice Chairman**, **Sri R.V.Srinivas** for his valuable support throughout the course.

I express our sincere thanks to **Dr.M.MOHAN BABU,** our beloved principal for his encouragement and suggestion during the course of study.

With the deep sense of gratefulness, I acknowledge **DR.D.SRIHARI,** Head of the Department, computer science and engineering(data science), for giving us inspiring guidance in undertaking internship.

I express our sincere thanks to the internship coordinator **Dr.J.SATHEESH KUMAR** , for her keen interest, stimulating guidance, constant encouragement with our work during all stages, to bring this report into fruition.

I wish to convey my gratitude and sincere thanks to all members for their support and coopera tion rendered for successful submission of report.

Finally, I would like to express my sincere thanks to all teaching, non-teaching faculty members, our parents, friends and for all those who have supported us to complete the internship successfully.

PERUGU DEVANAND
(Roll No:21781A04E8)

WEEKLY OVER VIEW OF INTERNSHIP ACTIVITIES

1stWEEK	DATE	DAY	NAMEOFTHETOPIC/MODULECOMPLETED
	01/02/2025	Saturday	Overview of java
	03/02/25	Monday	How to download java app
	04/02/25	Tuesday	First programming explain
	05/02/25	Wednesday	Variables and data types
	06/02/25	Thursday	Take user input and Operators
	07/0225	Friday	

2ndWEEK	DATE	DAY	NAMEOFTHETOPIC/MODULECOMPLETED
	10/02/25	Monday	Conditional statements
	11/02/25	Tuesday	Comments conversion
	12/02/25	Wednesday	Naming conversion
	13/02/25	Thursday	Overview of leveraging basic concepts
	14/02/25	Friday	Controls statements
	15/02/25	Saturday	Arrays

4th WEEK	DATE	DAY	NAMEOFTHETOPIC/MODULECOMPLETED
	24/02/25	Monday	Getter and Setter
	25/02/25	Tuesday	Inheritance
	26/02/25	Wednesday	Polymorphism
	27/02/25	Thursday	Method overloading
	28/02/25	Friday	Method and constructors overloading
	01/03/25	Saturday	Explain about various keywords

WEEKLY OVER VIEW OF INTERNSHIP ACTIVITIES

5th WEEK	DATE	DAY	NAMEOFTHETOPIC/MODULECOMPLETED
	03/03/25	Monday	Interface
	04/03/25	Tuesday	Collection framework
	05/03/25	Wednesday	File handling
	06/03/25	Thursday	Miscellaneous concepts
	07/03/25	Friday	Input /output: files
	08/03/25	Saturday	Stream classes

6th WEEK	DATE	DAY	NAMEOFTHETOPIC/MODULECOMPLETED
	10/03/25	Monday	Reading console input
	11/03/25	Tuesday	Threads: thread models
	12/03/25	Wednesday	Use of thread class
	13/03/25	Thursday	Project
	14/03/25	Friday	Project
	15/03/25	Saturday	Project

ABSTRACT

Java is a high-level, class-based, object-oriented programming language that is designed to have as few implementation dependencies as possible. It is a general-purpose programming language intended to let programmers write once, run anywhere meaning that compiled Java code can run on all platforms that support Java without the need to recompile. Java applications are typically compiled to bytecode that can run on any Java virtual machine (JVM) regardless of the underlying computer architecture. The syntax of Java is similar to C and C++, but has fewer low- level facilities than either of them. The Java runtime provides dynamic capabilities (such as reflection and runtime code modification) that are typically not available in traditional compiled languages. As of 2019, Java was one of the most popular programming languages in use according to GitHub, particularly for client–server web applications, with a reported 9 million developers.

Java was originally developed by James Gosling at Sun Microsystems. It was released in May 1995 as a core component of Sun Microsystems' Java platform. The original and reference implementation Java compilers, virtual machines, and class libraries were originally released by Sun under proprietary licenses. As of May 2007, in compliance with the specifications of the Java Community Process, Sun had relicensed most of its Java technologies under the GPL-2.0-only license. Oracle offers its own HotSpot Java Virtual Machine, however the official reference implementation is the OpenJDK JVM which is free open-source software and used by most developers and is the default JVM for almost all Linux distributions. In 1997, Sun Microsystems approached the ISO/IEC JTC 1 standards body and later the Ecma International to formalize Java, but it soon withdrew from the process. Java remains a de facto standard, controlled through the Java Community Process. At one time, Sun made most of its Java implementations available without charge, despite their proprietary software status. Sun generated revenue from Java through the selling of licenses for specialized products such as the Java Enterprise System.

S.no	CONTENTS	Page no
1.	Introduction…….	1
2.	Chapter- Java Terminology	
	2.1 Java Terminology ……. …	2 - 3
	2.2 Variables and Data types……...	3 - 5
	2.3 Operators of JAVA ……...……	6 - 14
3.	Chapter-2 Control Statements	
	3.1 Control Statement..……	15 - 23
	3.2 Arrays…….	24 - 26
	3.3 Strings..……..	27
4.	Chapter-3 Class And Object In JAVA	
	4.1 Class And Object In JAVA……..……	28 - 33
5.	Chapter-4 Extra contents of JAVA	
	5.1 Multithreading in Java……	34 - 38
	5.2 Generics in Java...……..	38 – 45

List of Tables

S.no	Name of the Table	Page No
1	Arithmetic Operators	6
2	Relational Operators	8
3	Assignment Operators	10
4	Logic Operators	14
5	Class and Objects	31

List of figures

S.no	Name of the Figure	Page No
1	Ternary	13
2	Array	24
3	Strings	27
4	Thread	35

Introduction

JAVA was developed by **James Gosling** at Sun Microsystems Inc in the year 1995, later acquired by Oracle Corporation. It is a simple programming language. Java makes writing, compiling, and debugging programming easy. It helps to create reusable code and modular programs. Java is a class-based, object-oriented programming language and is designed to have as few implementation dependencies as possible. A general-purpose programming language made for developers to write once run anywhere that is compiled Java code can run on all platforms that support Java. Java applications are compiled to byte code that can run on any Java Virtual Machine. The syntax of Java is similar to c/c++.

History: Java's history is very interesting. It is a programming language created in 1991. James Gosling, Mike Sheridan, and Patrick Naughton, a team of Sun engineers known as the Green team initiated the Java language in 1991. Sun Microsystems released its first public implementation in 1996 as Java 1.0. It provides no-cost -run-times on popular platforms. Java1.0 compiler was re-written in Java by Arthur Van Hoff to strictly comply with its specifications. With the arrival of Java 2, new versions had multiple configurations built for different types of platforms.In 1997, Sun Microsystems approached the ISO standards body and later formalized Java, but it soon withdrew from the process. At one time, Sun made most of its Java implementations available without charge, despite their proprietary software status. Sun generated revenue from Java through the selling of licenses for specialized products such as the Java Enterprise System.On November 13, 2006, Sun released much of its Java virtual machine as free, open-source software. On May 8, 2007, Sun finished the process, making all of its JVM's core code available under open-source distribution terms.

CHAPTER 1 Java Terminology

Before learning Java, one must be familiar with these common terms of Java.

1. **Java Virtual Machine(JVM):** This is generally referred to as JVM. There are three execution phases of a program. They are written, compile and run the program.Writing a program is done by a java programmer like you and me.The compilation is done by the JAVAC

compiler which is a primary Java compiler included in the Java development kit (JDK). It takes the Java program as input and generates bytecode as output.In the Running phase of a program, JVM executes the bytecode generated by the compiler.

Now, we understood that the function of Java Virtual Machine is to execute the bytecode produced by the compiler. Every Operating System has a different JVM but the output they produce after the execution of bytecode is the same across all the operating systems. This is why Java is known as a platform-independent language.

2. **Bytecode in the Development process:** As discussed, the Javac compiler of JDK compiles the java source code into bytecode so that it can be executed by JVM. It is saved as .class file by the compiler. To view the bytecode, a disassembler like javap can be used.

3. **Java Development Kit(JDK):** While we were using the term JDK when we learn about bytecode and JVM. So, as the name suggests, it is a complete Java development kit that includes everything including compiler, Java Runtime Environment (JRE), java debuggers, java docs, etc. For the program to execute in java, we need to install JDK on our computer in order to create, compile and run the java program.

4. **Java Runtime Environment (JRE):** JDK includes JRE. JRE installation on our computers allows the java program to run, however, we cannot compile it. JRE includes a browser, JVM, applet supports, and plugins. For running the java program, a computer needs JRE.

5. **Garbage Collector:** In Java, programmers can't delete the objects. To delete or recollect that memory JVM has a program called Garbage Collector. Garbage Collectors can recollect the objects that are not referenced. So Java makes the life of a programmer easy by handling memory management. However, programmers should be careful about their code whether they are using objects

that have been used for a long time. Because Garbage cannot recover the memory of objects being referenced.

6. ClassPath: The classpath is the file path where the java runtime and Java compiler look for .class files to load. By default, JDK provides many libraries. If you want to include external libraries they should be added to the class path.

MY FIRST JAVA APPLICATION : public
class Main {

public static void main(String args[]){

System.out.println("Hello World");

}

}

OUT PUT

Hello World

Variables and Data types:

- String - stores text, such as "Hello". ...
- int - stores integers (whole numbers), without decimals, such as 123 or -123.
- float - stores floating point numbers, with decimals, such as 19.99 or -19.99.
- char - stores single characters, such as 'a' or 'B'. ...
- boolean - stores values with two states: true or false.

Data types are different sizes and values that can be stored in the variable that is made as per convenience and circumstances to cover up all test cases. Also, let us cover up other important ailments that there are majorly two types of languages that are as follows:

1. First, one is a **Statically typed language** where each variable and expression type is already known at compile time. Once a variable is declared to be of a certain data type, it cannot hold values of other data types. For example C, C++, Java.

2. The other is **Dynamically typed languages.** These languages can receive different data types over time.

class Guru99 {

static int a = 1; //static variable int

data = 99; //instance variable

```
    void method() {
        int b = 90; //local variable

    }
}
```

USER INPUT :

There are several ways to get user input in java. The Scanner class is used to take user input, and this scanner class is present in the java. until package. For using the object of Scanner, we have to import java. until .Scanner package. like: import java .until. Scanner; Then, we have to create an object of the Scanner class for taking input from the user

```
// creating an object of Scanner

Scanner input = new Scanner(System.in);

// taking input from the user

int number = input.nextInt();
```

example :

```
import java.util.Scanner;

class Input {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter an integer: "); int
        numb = input.nextInt();
        if(numb<100){
        System.out.println("You entered " + numb);
} else{
```

```
    System.out.println("You entered " + numb +" invalid number.");

}

        // closing the scanner object

        input.close();

    }

}
```

Taking string input :

The nextLine() method of Scanner class is used to take a string from the user. It is defined in java.util.Scanner class. The nextLine() method reads the text until the end of the line. After reading the line, it throws the cursor to the next line.

```
import java.util.*;

class UserInputDemo1

{

public static void main(String[] args)

{

Scanner sc= new Scanner(System.in); //System.in is a standard input stream

System.out.print("Enter a string: ");

String str= sc.nextLine();              //reads string

System.out.print("You have entered: "+str);

}

}
```

Operators of JAVA :

There are many types of operators in Java which are given below

- Arithmetic Operator,
- Relational Operator,
- Increment and Decrement operator,
- Logical Operator,
- Ternary Operator and
- Assignment Operator.

Arithmetic, Increment and Decrement Operator :

- ✞ Addition
- ✞ Subtraction
- ✞ Multiply
- ✞ Divide
- ✞ Modulus

Table 1

Operators	Result
+	Addition of two numbers
-	Subtraction of two numbers
*	Multiplication of two numbers
/	Division of two numbers
%	(Modulus Operator)Divides two numbers and returns the remainder
++	Increment Operator
--	Decrement Operator

Example :

```
int i = 37;
    int j = 42;
double x = 27.475;
```

```
        double y = 7.22;
        System.out.println("Variable values...");
        System.out.println("    i = " + i);
        System.out.println("    j = " + j);
        System.out.println("    x = " + x);
        System.out.println("    y = " + y);

        //adding numbers
        System.out.println("Adding...");
        System.out.println(" i + j = " + (i + j));
        System.out.println(" x + y = " + (x + y));

        //subtracting numbers
        System.out.println("Subtracting...");
        System.out.println(" i - j = " + (i - j)); System.out.println(" x
        - y = " + (x - y));

        //multiplying numbers
        System.out.println("Multiplying...");
        System.out.println(" i * j = " + (i * j));
        System.out.println(" x * y = " + (x * y));

        //dividing numbers
        System.out.println("Dividing...");
        System.out.println(" i / j = " + (i / j));
        System.out.println(" x / y = " + (x / y));

        //computing the remainder resulting from dividing numbers
        System.out.println("Computing the remainder...");
        System.out.println(" i % j = " + (i % j));
        System.out.println(" x % y = " + (x % y));

        //mixing types
        System.out.println("Mixing types...");
        System.out.println(" j + y = " + (j + y));
        System.out.println(" i * x = " + (i * x));
    } }
```

The output from this program is: Variable values...

i = 37 j = 42

x = 27.475

y = 7.22

Adding... i + j = 79

x + y = 34.695

Subtracting... i - j = -5

x - y = 20.255

Multiplying...

i * j = 1554 x * y = 198.37

Dividing... i / j = 0

x / y = 3.8054

Computing the remainder...

i % j = 37 x % y = 5.815

Mixing types... j + y = 49.22

i * x = 1016.58

Relational Operators:

Relational operators are a bunch of binary operators used to check for relations between two operands, including equality, greater than, less than, etc. They return a boolean result after the comparison and are extensive y used in looping statements as well as conditional ielse statements and so on

Relational Operators

Operators	Meaning	Example	Result
<	Less than	5<2	False
>	Greater than	5>2	True
<=	Less than or equal to	5<=2	False
>=	Greater than or equal to	5>=2	True
==	Equal to	5==2	False
! =	Not equal to	5! =2	True
===	Equal value and same type	5 === 5	True
		5 === "5"	False
! ==	Not Equal value or Not same type	5 ! == 5	False
		5 ! == "5"	True

Table 2

Example:

```
// Java code to Illustrate Less than Operator


// Importing I/O classes
import java.io.*;

// Main class
class GFG {

    // Main driver method public static
    void main(String[] args)
    {
            // Initializing variables int var1
            = 10, var2 = 20, var3 = 5; //
            Displaying var1, var2, var3
            System.out.println("Var1 = " + var1);
            System.out.println("Var2 = " + var2);
            System.out.println("Var3 = " + var3);
            // Comparing var1 and var2 and
            // printing corresponding boolean value
            System.out.println("var1 < var2: " + (var1 < var2));
            // Comparing var2 and var3 and
            // printing corresponding boolean value
            System.out.println("var2 < var3: " + (var2 < var3));
    }

}
```

Out put:

- Var1 = 10
- Var2 = 20
- Var3 = 5
- var1 < var2: true
- var2 < var3: false

Assignment operators :

Java's basic assignment operator is a single equals-to (=) sign that assigns the right-hand

Table 3

Assignment Operators

Operator	Example	Equivalent Expression
=	$m = 10$	$m = 10$
+=	$m += 10$	$m = m + 10$
−=	$m -= 10$	$m = m - 10$
*=	$m *= 10$	$m = m * 10$
/=	$m /=$	$m = m/10$
%=	$m \% = 10$	$m = m\%10$
<<=	$a <<= b$	$a = a << b$
>>=	$a >>= b$	$a = a >> b$
>>>=	$a >>>= b$	$a = a >>> b$
&=	$a \& = b$	$a = a \& b$
^=	$a \hat{} = b$	$a = a \hat{} b$
|=	$a \mid = b$	$a = a \mid b$

side value to the left-hand side operand. On left side of the assignment operator there must be a variable that can hold the value assigned to it. Assignment operator operates on both primitive and reference types. **Example:**

```
class AssignmentOperator { public static
   void main (String[] args) { int a = 2; int
   b = a; int c = a + b; int d = sum(a,b);
   boolean e = a>b;
      System.out.println("a = " + a);
      System.out.println("b = " + b);
      System.out.println("c = " + c);
      System.out.println("d = " + d);
```

```
    System.out.println("e = " + e);
  } static int sum(int x, int y)
  {
    return x+y;
  }
 }
```

Out put:

a=2 b=2
c=4 d=4
e = false

Logical operator :

This operator returns true when one of the two conditions under consideration is satisfied or is true. If even one of the two yields true, the operator results true. To make the result false, both the constraints need to return false.

Table 4

Logical Operators

Example	Name	Result
$a and $b	And	**TRUE** if both *$a* and *$b* are **TRUE**.
$a or $b	Or	**TRUE** if either *$a* or *$b* is **TRUE**.
$a xor $b	Xor	**TRUE** if either *$a* or *$b* is **TRUE**, but not both.
! $a	Not	**TRUE** if *$a* is not **TRUE**.
$a && $b	And	**TRUE** if both *$a* and *$b* are **TRUE**.
$a || $b	Or	**TRUE** if either *$a* or *$b* is **TRUE**.

Example:

```
// Java code to illustrate
// logical AND operator
import java.io.*; class
Logical {
   public static void main(String[] args)
```

```
    {

        // initializing variables int a =
        10, b = 20, c = 20, d = 0; //
        Displaying a, b, c
        System.out.println("Var1 = " + a);
        System.out.println("Var2 = " + b);
        System.out.println("Var3 = " + c);
          // using logical AND to verify
        // two constraints if ((a < b)
        && (b == c)) {
            d = a + b + c;
            System.out.println("The sum is: " + d);

        } else
            System.out.println("False conditions");
    }
}
```

Out put:

Var1 = 10

Var2 = 20

Var3 = 20

The sum is: 50

Ternary operator:

Java ternary operator is the only conditional operator that takes three operands. It's a one-liner replacement for the if-then-else statement and is used a lot in Java programming. We can use the ternary operator in place of if-else conditions or even switch conditions using nested ternary

operators. Although it follows the same algorithm as of if-else statement, the conditional operator takes less space and helps to write the if-else statements in the shortest way possible.The syntax of the ternary operator is given as follows

Expression ? Statement 1 : Statement 2In the above syntax, the expression is a conditional expression that results in true or false. If the value of the expression is true, then statement 1 is executed otherwise statement 2 is executed.

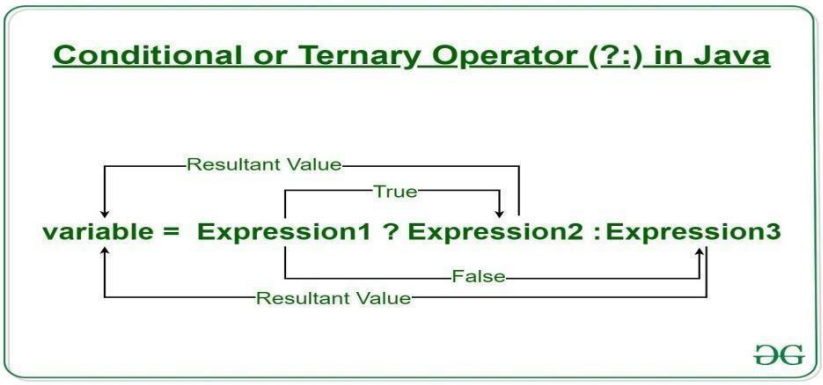


Figure 1

Example:

```
// Java program to find largest among two

// numbers using ternary operator import

java.io.*;

class Ternary {

    public static void main(String[] args)

    {
        // variable declaration int
        n1 = 5, n2 = 10, max;
        System.out.println("First num: " + n1);
        System.out.println("Second num: " + n2);
        // Largest among n1 and n2
```

```
        max = (n1 > n2) ? n1 : n2;
        // Print the largest number
        System.out.println("Maximum is = " + max);

}
}
```

CHAPTER 2

Control Statements : As statemnts give instructions to the program, control statements constraints the flow of the program. Control statements are categorized as

1. Decision-Making statements In Java

- If statement in Java
- If-else statement in Java
- Nested if statement in Java
- Nested if-else statement in Java
- Switch statement in Java

2. Looping statements In Java

- while loop in Java
- do-while loop in Java
- for loop in Java

3. Branching statements In Java

- The break statement in Java
- The continue statement in Java
- The return statement in Java

If statement: This statement will execute the following block of code if the given condition is satisfied else it skips the if block of code **Syntax:**

```
if (condition)
{
statement;   // will be executed if the condition is true, else skips to next statement
} statement;   // will be executed irrespective of the condition
```

Example:

```
class IfExp
{ public static void main(String args[])
  { int x = 4;
    if (x > 10)
       System.out.println("If condition is true");
    System.out.println("After if block"); }
}
```

If-else statement: Adding an else block in addition to if block will execute the else block only if the if block condition is not true.

Syntax:

```
if (condition a) {
    Statement a;          //executed when condition a is true
} else
{
   Statement b;   //executed when condition a is false }
```

Example:

```
class IfElseExp {
public static void main(String args[]) {

   int a = 19; if
   (a > 100)
     System.out.println("a is bigger than 100"); else
     System.out.println("a is less than 100");
  System.out.println("After if else statements"); }
}
```

Nested if statement: The nested if statement will be executed if the condition is true and the parent if the condition is true.

Syntax:

```
if (condition a) {
    Statement a; //executed when condition a is true if
    (condition b) {
        Statement b;   //executed when condition b is true } else {
        Statement c;   //executed when condition b is false
    }
}
```

Example:

```
class IfElseExp {
public static void main(String args[]) {
   int a = 55;
if (a > 13) {
```

```
    if (a % 2 == 0)
      System.out.println("a is bigger than 13 and is even"); else
      System.out.println("a is odd and bigger than 13");
   } else {
     System.out.println("a is less than 13");
   }
  System.out.println("After  nested  statements"); }
}
```

Nested if-else statement: The nested if-else statements will be required when you need to test for more than 2 conditions.

Syntax:

```
if (condition a) {

   Statement a; //will be executed if condition a is true} else

if (condition b) {

  Statement b; //will be executed if condition b is true


}

else {

 Statement c; //executed when none of the conditions is true }
```

Example:

```
lass NEstIfElseExp{
public static void main(String args[]) {
   int a = 23; if
   (a > 23) {
    System.out.println("a is bigger than 23"); }
   else if (a < 23) {
     System.out.println("a is lesser than 23");
   } else {
     System.out.println("a is 23");
   }
  System.out.println("After if else if blocks"); }
}
```

Switch statement: You can rewrite and define those nested if-else statements more clearly with a switch statement. It starts with an expression passed to switch and checks for various values with the case. Multiple cases should not have similar values nor should they be of different data types as of expression. A break statement can be used after any case, to cease the execution of the next cases.

Syntax:

```
switch (expression)
{
  case value a:
  Statement a;
  break; case
  value b:
  Statement b;
  break; case
  value n:
  Statement n;
  break;
  default:
    Default statement;}
```

Example:

```
class SwitchExp {
public static void main(String args[]) {
  int a = 7; switch (a) { case 7:
     System.out.println("a is seven."); break;
   case 2:
```

```
    System.out.println("a is two."); break;
  case 3:
    System.out.println("a is three."); break;
  case 4:
    System.out.println("a is four."); break;
  case 1:
    System.out.println("a is one.");
  break; default:

    System.out.println("a is greater than 4.");

  }
}

}
```

while loop

The compiler will execute the while block until the passed condition is true, or if the condition of the while loop is false it will never be executed. A boolean condition should be passed to the while loop.

Syntax:

```
while (condition)
{
Statement;
}
```

Example:

```
class WhileExp{

public static void main(String args[]) {

  int i = 1;

  while (i <= 23) {

    System.out.println(i); i

    = i + 3;

  }

}

}
```

do-while loop: This is similar to while loop except that the condition of the while loop condition will be checked only after the block of code is executed at least once

Syntax: do{

statement

```
}while(condition);
```

Example:

```
class Main {

public static void main(String args[]) {

  int a = 17; do {

     System.out.println(a); a

     = a + 2;

   } while (a <= 17);
}
}
```

for loop: for loops are used to repeat a certain task specified number of times along with a condition.It consists of an initialization value, a condition, and increment or decrement of value.

Syntax:

```
for (initialization; condition; increment/decrement)
{

    statement;
}
```

Example:

```
class Main {

public static void main(String args[]) { for

  (int a = 3; a <= 15; a+=3)

  System.out.println(a);


}
```

}

The break statement: This statement is used to terminate the execution of looping statements after the break keyword.

Example:

```
class BreakExp{
public static void main(String args[]) { int b

  = 0;

   for (int a = 0; a < 10; a++) {

     // come out of loop when j is 5 if

     (a == 5) {

       System.out.println("test break here"); break;

     }

     System.out.println("a: " + a);

   }

   System.out.println("After for loop");

   while (b < 10) {
     b += 2;
     // come out of loop when b is greater than 4 if
     (b >= 5) {
       System.out.println("test break here"); break;
     }
     System.out.println("b: " + b); }
  System.out.println("After while loop"); }
}
```

The continue statement: This statement is used to skip a particular iteration statement and jump to the next iteration.

Example:

```
class ContinueExp{
public static void main(String args[]) {
   for (int i = 1; i < 15; i++) {
     // If the number is even then bypass and continue with next iteration if (i
     % 2 == 0)
       continue;
     // printing only odd numbers
     System.out.print(i + " ");
   }
}
}
```

The return statement: The return statements are usually used at the end of the function or where you want to bypass the function. The return keyword is used to return the estimated value from the function to the calling function **Example:**

```
public class Main {
static int myMethod(int x) {
   return 10 + x;
}


public static void main(String[] args) {
System.out.println(myMethod(13)); }
}
```

Arrays

Normally, an array is a collection of similar type of elements which has contiguous memory location.

In Java, array is an object of a dynamically generated class. Java array inherits the Object class, and implements the Serializable as well as Cloneable interfaces. We can store primitive values or objects in an array in Java. Like C/C++, we can also create single dimentional or multidimentional arrays in Java.Moreover, Java provides the feature of anonymous arrays which is not available in C/C++.

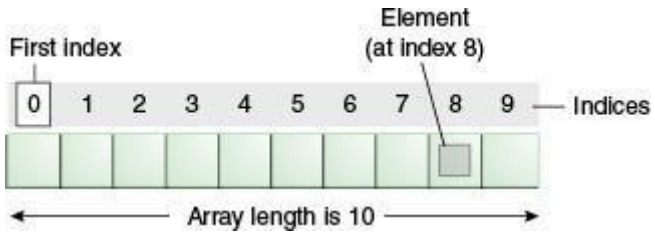


Figure 2

Advantages

- **Code Optimization:** It makes the code optimized, we can retrieve or sort the data efficiently.
- **Random access:** We can get any data located at an index position.

Disadvantages

- **Size Limit:** We can store only the fixed size of elements in the array. It doesn't grow its size at runtime. To solve this problem, collection framework is used in Java which grows automatically.

Types of Array in java

There are two types of array. ○

Single Dimensional Array

- Multidimensional Array

Single Dimensional Array

Syntax to Declare an Array in Java

1. dataType[] arr; (or)
2. dataType []arr; (or)
3. dataType arr[];

Instantiation of an Array in Java

1. arrayRefVar=**new** datatype[size];

Example

```
//Java Program to illustrate how to declare, instantiate, initialize //and
traverse the Java array.
class Testarray{ public static void main(String
args[]){ int a[]=new int[5];//declaration and
instantiation a[0]=10;//initialization a[1]=20;
a[2]=70; a[3]=40; a[4]=50; //traversing array
for(int i=0;i<a.length;i++)//length is the property of array
System.out.println(a[i]);
}}
```

Multidimensional Array

In such case, data is stored in row and column based index (also known as matrix form).

Syntax to Declare Multidimensional Array in Java

1. dataType[][] arrayRefVar; (or)
2. dataType [][]arrayRefVar; (or)
3. dataType arrayRefVar[][]; (or)
4. dataType []arrayRefVar[];

Example to instantiate Multidimensional Array in Java

```
1. int[][] arr=new int[3][3];//3 row and 3 column or
1.  arr[0][0]=1;
2.  arr[0][1]=2;
3.  arr[0][2]=3;
4.  arr[1][0]=4;
5.  arr[1][1]=5;
6.  arr[1][2]=6;
7.  arr[2][0]=7;
8.  arr[2][1]=8;
9.  arr[2][2]=9;
```

Example of Multidimensional

Let's see the simple example to declare, instantiate, initialize and print the 2Dimensional array.

```
//Java Program to illustrate the use of multidimensional array
class Testarray3{ public static void main(String args[]){
//declaring and initializing 2D array int
arr[][]={{1,2,3},{2,4,5},{4,4,5}};
//printing 2D array for(int
i=0;i<3;i++){ for(int
j=0;j<3;j++){
   System.out.print(arr[i][j]+" ");
}
System.out.println();
}
}}
```

Strings

Strings are used for storing text.

A String variable contains a collection of characters surrounded by double quotes: **Example**

Create a variable of type String and assign it a value:

String greeting = "Hello";

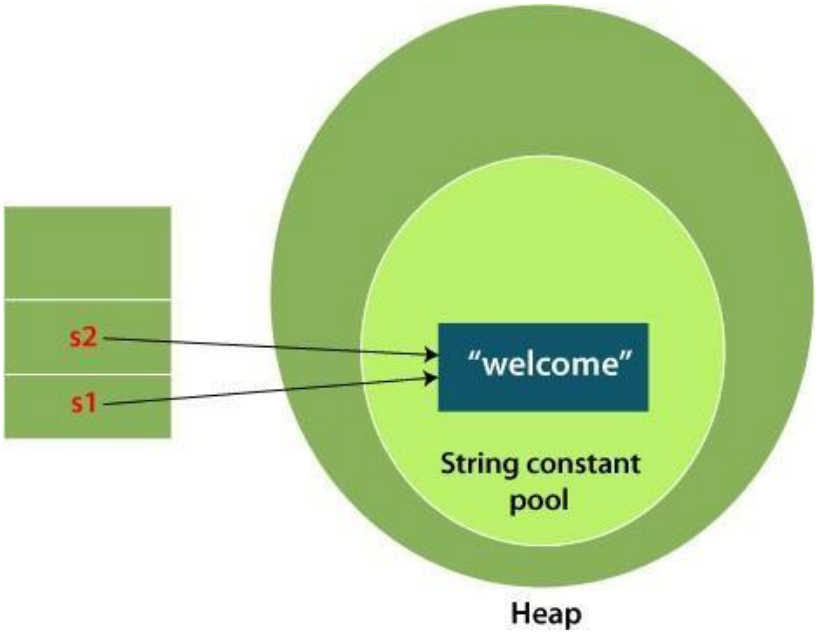
String Length

A String in Java is actually an object, which contain methods that can perform certain operations on strings. For example, the length of a string can be found with the length() method:

Example

String txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";

System.out.println("The length of the txt string is: " + txt.length());



F

CHAPTER 3

Class And Object In JAVA:

JAVA is an Object Oriented Programming which simply means coding in JAVA constantly involve classes and objects. In other words coding in JAVA is not possible without object and class. Even the smallest Hello world program requires declaration of class and method work on object.

Class: The concept of class comes into role when we see certain type of objects or things around us and the common idea or a blueprint behind this type of objects is called Class. In other words class is a properties behind each of the objects or things possess.

For example: Consider you have iPhone, Samsung and Sony devices and you want to represent them in JAVA. To do that you first need to find out what can be the blueprint behind these devices. And here blueprint can be a Mobile because they all are a type of Mobile. So Mobile is a class which can represent iPhone, Samsung and Sony devices here.

```
class Mobile{

/**ToDo Code Here*/

}
```

Class is a representation of similar types of objects or it is a implementation of encapsulation.

- In terms of java, a class is type declaration means when you want to define a specific type of data for special use then it can be easily obtained with the help of class.
- Basically, class is blueprint for specific type. In the human language across the world, if you count, there are lots of classes e.g. car, bank, bird, student, employee etc. are very simple example to understand.

- Class is created with the word class when you want to define it. Ex: class Mobile{}
- Class itself consists of various methods and variable.

- To call upon class objects of other classes there must be main method with static keyword. Because with the static word method will be called even if you do not create its objects.
- Classes are provided with special access modifiers that are **default, public, private** and **protected**.

Object: Object is an instance of class. Understanding the concept of object is lot easier when considering real life examples around us because the concept is actually based on real life objects. So just look around yourself and you will find yourself surrounded with lots of objects which has a certain characteristics and behaviors.

```
//Class Mobile class

Mobile{ String

brand, color; int

camera;

//Constructor initialized
public Mobile(String brand, String color, int camera){

this.brand = brand; this.color = color; this.camera =

camera;

}

public static void main(String args[]){

//Object created
```

```
Mobile abhishek = new Mobile("iPhone","Gold",8);

        Mobile rahul = new Mobile("Samsung","White",13);

        Mobile ravi = new Mobile("Nexus","Black",8);

    //Smartphone details displayed for each user
        System.out.println("Abhishek own " + abhishek.brand +" "+ abhishek.color + " color
    smartphone having "+ abhishek.camera+ "MP"+ " camera");

        System.out.println("Rahul own " + rahul.brand +" "+ rahul.color + " color smartphone having
    "+ rahul.camera+ "MP"+ " camera");

        System.out.println("Ravi own " + ravi.brand +" "+ ravi.color + " color smartphone having "+
    ravi.camera+ "MP"+ " camera");

    }

      }
```

Important Points about class and object:

The word “Class” came from simula language.

Class is blueprint for an entity. In class

there are variables and methods.

The access modifiers like public, private and protected used in different situation.
Objects represent the state and behavior of class.
Object is just a memory area or a buffer in heap area in which all the instance data members are getting a memory.

In Java “new” operator is only used for allocating a memory for an object only.

Memory associated with object is automatically collected by garbage collector.

Class with main method having static keyword is mandatory to call upon the object of other classes.

All the objects are sharing a same memory location for each static data members.

Apache Tika is used for document type detection and content extraction from various file formats. It uses various document parsers and document type detection techniques to detect and extract data. It provides a single generic API for parsing different file formats. All these parser libraries are encapsulated in a single interface called the Parser interface. The following table shows a description of the important methods used in the solution :

BodyContentHandler(	It creates a content handler that writes XHTML body character events to an
Metadata()	It constructs new empty meta data
ParseContext()	It creates a parse context object that is used to pass context information to Tika parsers
parse()	Instantiate the parser object, and invoke the parse method.

Example: Java code to extract the contents of Java class file format

```
// Java program to extract the
// contents of Java class file
// format
  import java.io.File;
import java.io.FileInputStream; import
java.io.IOException;
  // importing Apache Tika libraries
```

`

```
import org.apache.tika.exception.TikaException;
import org.apache.tika.metadata.Metadata;
import org.apache.tika.parser.AutoDetectParser;
import org.apache.tika.parser.ParseContext;
import org.apache.tika.parser.Parser; import
org.apache.tika.sax.BodyContentHandler;
import org.xml.sax.SAXException;
 public class ParserExtraction {
    public static void main(final String[] args)
      throws IOException, SAXException, TikaException
  {
      // create a File object
      File f = new File("AddTwoNumbers.java");
       // parse method parameters
      Parser parser = new AutoDetectParser();
       // instantiate BodyContentHandle
      BodyContentHandler handler
        = new BodyContentHandler();
       // Creates the Metadata object
      Metadata metadata = new Metadata();
      FileInputStream inputstream
        = new FileInputStream(f);
```

```
        // creates a parse context object
        ParseContext context = new ParseContext();
        // parsing the file
        parser.parse(inputstream, handler, metadata, context);
                // display the file content
        System.out.println("File content : "
                        + Handler.toString());
    }
}
```

Output:

```
File content : public class AddTwoNumbers {
  public static void main(String[] args) {
     int num1 = 5, num2 = 15, sum; sum =
     num1 + num2;
     System.out.println("Sum of these numbers: "+sum);
  }
```

CHAPTER-4

Multithreading in Java

1. Multithreading

`

2. Multitasking
3. Process-based multitasking
4. Thread-based multitasking
5. What is Thread

Multithreading in Java is a process of executing multiple threads simultaneously.

A thread is a lightweight sub-process, the smallest unit of processing. Multiprocessing and multithreading, both are used to achieve multitasking.

However, we use multithreading than multiprocessing because threads use a shared memory area. They don't allocate separate memory area so saves memory, and context-switching between the threads takes less time than process.

Java Multithreading is mostly used in games, animation, etc.

Advantages of Java Multithreading

1) It doesn't block the user because threads are independent and you can perform multiple operations at the same time.
2) You can perform many operations together, so it saves time.
3) Threads are independent, so it doesn't affect other threads if an exception occurs in a single thread.

Multitasking

Multitasking is a process of executing multiple tasks simultaneously. We use multitasking to utilize the CPU. Multitasking can be achieved in two ways:

- Process-based Multitasking (Multiprocessing)
- Thread-based Multitasking (Multithreading)

1) Process-based Multitasking (Multiprocessing)
   - Each process has an address in memory. In other words, each process allocates a separate memory area.
   - A process is heavyweight.
   - Cost of communication between the process is high.
   - Switching from one process to another requires some time for saving and loading registers, memory maps, updating lists, etc.
2) Thread-based Multitasking (Multithreading)
   - Threads share the same address space.
   - A thread is lightweight.

- Cost of communication between the thread is low.

What is Thread in java

A thread is a lightweight subprocess, the smallest unit of processing. It is a separate path of execution.

Threads are independent. If there occurs exception in one thread, it doesn't affect other threads. It uses a shared memory area**.**

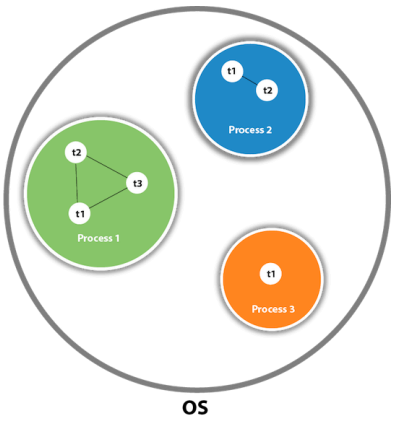


Figure 4

As shown in the above figure, a thread is executed inside the process. There is context-switching between the threads. There can be multiple processes inside the OS, and one process can have multiple threads.

Note: At a time one thread is executed only.

Exception Handling in Java

Exception Handling in Java is a distinctive approach to improvise a Java application's convenience and performance capabilities. Exceptions, if not handled properly, may pose a severe threat to the application program in real-time.

Types of Exceptions

There are technically two types of Exceptions, and the third variety is an error. Those are mentioned below as follows.

- Checked Exceptions
- Unchecked Exceptions

`

- Errors

Checked Exceptions

The classes that inherit all the exceptions from the throwable parent class directly, but except for the run-time exception, are called the checked exceptions.

Example:

SQL Exception and IOException //code

1:

```
//Checked exception (unresolved)
package Exceptions; import
java.io.*; public class Checked {
public static void main(String[] args) {
FileReader file = new FileReader("C:\\Users\\ravi.kiran\\Documents\\data.txt");
BufferedReader Input = new BufferedReader(file); for (int c = 0; c < 3; c++)
System.out.println(Input.readLine());
Input.close();
}
}
```

//code 2:

```
//Checked exception (resolved)
package Exceptions; import
java.io.*; public class Checked
{
public static void main(String[] args) throws IOException {
FileReader file = new FileReader("C:\\Users\\ravi.kiran\\Documents\\data.txt");
BufferedReader Input = new BufferedReader(file); for
(int c = 0; c < 5; c++)
System.out.println(Input.readLine());
Input.close();
}
}
```

Unchecked Exceptions

The classes that inherit only the run-time exceptions are called as the Unchecked Exceptions.

Example:

ArithmeticException and NullPointerException //code:

Unchecked Exceptions(Resolved)

```
package Exceptions; public class
Unchecked { public static void
main(String args[]) { int Ary[] = { 1, 2,
3, 4, 5, 6, 7, 8, 9, 10 };
System.out.println(Ary[11]);
}
}
```

Methods for Exception Handling in Java

There are two ways for Exception Handling in Java. They are as follows.

- Default Exception Handling in Java
- User-Defined Exception Handling in Java

Default Exception Handling in Java

Java Virtual machine handles default exceptions. The compiler identifies the presence of an exception, it quickly packs the recognized exception in the form of an object.

The compiler sends the exception object to the JVM during the run-time. This process is called throwing an Exception.

After the exception is thrown, many methods could get executed in response to the thrown exception. The list is known as the Call Stack. The Call Stack is described as follows.

- The thrown exception reaches the call stack, and the call stack responds with an exception handler that could handle the thrown exception.
- If the run-time system fails to recognize the appropriate exception handler for the thrown exception, then the run-time system sends the exception object to the default exception handler.
- The default exception handler results in an abnormal output that reads out a report related to the bizarre exception encounter.

User-Defined Exception Handling in Java

The customized/user-defined exception handling in Java is managed by using the exception handling keywords. They are:

`

- try
- catch
- throw
- throws
- finally

In customized exception handling, the user should recognize/expect an exception at a specific part of the code segment.

Once the location of the exception is finalized, then, the code block should be enclosed inside the try and catch blocks.

With this setup, whenever the code throws an exception, it gets handled by the appropriate catch block.

The "throw" keyword is used to manually throw an exception. When an exception is thrown from the program, it is identified by the "throws" clause.

Generics in Java

Generics means **parameterized types**. The idea is to allow type (Integer, String, … etc., and user-defined types) to be a parameter to methods, classes, and interfaces. Using Generics, it is possible to create classes that work with different data types. An entity such as class, interface, or method that operates on a parameterized type is a generic entity.

Why Generics?

The **Object** is the superclass of all other classes, and Object reference can refer to any object. These features lack type safety. Generics add that type of safety feature. We will discuss that type of safety feature in later examples.

Generics in Java are similar to templates in C++. For example, classes like HashSet, ArrayList, HashMap, etc., use generics very well. There are some fundamental differences between the two approaches to generic types.

Types of Java Generics

Generic Method: Generic Java method takes a parameter and returns some value after performing a task. It is exactly like a normal function, however, a generic method has type parameters that are cited by actual type. This allows the generic method to be used in a more general way. The compiler takes care of the type of safety which enables programmers to code easily since they do not have to perform long, individual type castings.

Generic Classes: A generic class is implemented exactly like a non-generic class. The only difference is that it contains a type parameter section. There can be more than one type of parameter, separated by a comma. The classes, which accept one or more parameters, are known as parameterized classes or parameterized types.

Generic Class

Like C++, we use <> to specify parameter types in generic class creation. To create objects of a generic class, we use the following syntax.

// To create an instance of generic class

BaseType <Type> obj = new BaseType <Type>() **Generic**

Functions:

We can also write generic functions that can be called with different types of arguments based on the type of arguments passed to the generic method. The compiler handles each method.

- Java

```
// Java program to show working of user defined
// Generic functions


class Test {
    // A Generic method example static <T>
    void genericDisplay(T element)
    {
        System.out.println(element.getClass().getName()


                           + " = " + element);
    }


    // Driver method public static void
    main(String[] args) {
        // Calling generic method with Integer argument genericDisplay(11);
```

`

```
        // Calling generic method with String argument genericDisplay("GeeksForGeeks");

        // Calling generic method with double argument genericDisplay(1.0);
    }
}
```

Output

java.lang.Integer = 11

java.lang.String = GeeksForGeeks

java.lang.Double = 1.0 **Generics**

Work :

When we declare an instance of a generic type, the type argument passed to the type parameter must be a reference type. We cannot use primitive data types like **int**, **char.**

Test<int> obj = new Test<int>(20);

The above line results in a compile-time error that can be resolved using type wrappers to encapsulate a primitive type.

But primitive type arrays can be passed to the type parameter because arrays are reference types.

ArrayList<int[]> a = new ArrayList<>();

Generic Types Differ Based on Their Type Arguments:

Consider the following Java code.

- Java

```
// Java program to show working
// of user-defined Generic classes

// We use < > to specify Parameter type class
Test<T> {
    // An object of type T is declared
    T obj;
    Test(T obj) { this.obj = obj; } // constructor public
    T getObject() { return this.obj; }
```

```
}

// Driver class to test above class
Main {
    public static void main(String[] args)
    {
        // instance of Integer type
        Test<Integer> iObj = new Test<Integer>(15);
        System.out.println(iObj.getObject());

        // instance of String type
        Test<String> sObj
            = new Test<String>("GeeksForGeeks");
        System.out.println(sObj.getObject()); iObj
        = sObj; // This results an error
    }
}
```

Output: error:
incompatible types:

Test cannot be converted to Test
Even though iObj and sObj are of type Test, they are the references to different types because their type parameters differ. Generics add type safety through this and prevent errors.

Type Parameters in Java Generics

The type parameters naming conventions are important to learn generics thoroughly. The common type parameters are as follows:

- T – Type
- E – Element
- K – Key

`

- N – Number
- V – Value

Advantages of Generics:

Programs that use Generics has got many benefits over non-generic code.

1. Code Reuse: We can write a method/class/interface once and use it for any type we want.

2. Type Safety: Generics make errors to appear compile time than at run time (It's always better to know problems in your code at compile time rather than making your code fail at run time). Suppose you want to create an ArrayList that store name of students, and if by mistake the programmer adds an integer object instead of a string, the compiler allows it. But, when we retrieve this data from ArrayList, it causes problems at runtime.

- Java

```
// Java program to demonstrate that NOT using //
generics can cause run time exceptions

import java.util.*;

class Test
{ public static void main(String[] args)
   {
      // Creatinga an ArrayList without any type specified
      ArrayList al = new ArrayList();


      al.add("Sachin"); al.add("Rahul");
      al.add(10); // Compiler allows this


      String s1 = (String)al.get(0);
      String s2 = (String)al.get(1);

      // Causes Runtime Exception String
      s3 = (String)al.get(2);
   }
```

```
}
```

Output :

```
Exception in thread "main" java.lang.ClassCastException:
  java.lang.Integer cannot be cast to java.lang.String at
  Test.main(Test.java:19)
```

How do Generics Solve this Problem?

When defining ArrayList, we can specify that this list can take only String objects.

- Java

```
// Using Java Generics converts run time exceptions into //
compile time exception.
import java.util.*;

class Test
{ public static void main(String[] args)
  {
    // Creating a an ArrayList with String specified
    ArrayList <String> al = new ArrayList<String> ();


    al.add("Sachin");
    al.add("Rahul");


    // Now Compiler doesn't allow this al.add(10);


    String s1 = (String)al.get(0);
    String s2 = (String)al.get(1);
    String s3 = (String)al.get(2);
  }
}
```

Output:

`

15: error: no suitable method found for add(int) al.add(10);

^

3. Individual Type Casting is not needed: If we do not use generics, then, in the above example, every time we retrieve data from ArrayList, we have to typecast it. Typecasting at every retrieval operation is a big headache. If we already know that our list only holds string data, we need not typecast it every time.

- Java

```
// We don't need to typecast individual members of ArrayList

import java.util.*;

class Test { public static void
   main(String[] args)
   {
      // Creating a an ArrayList with String specified
      ArrayList<String> al = new ArrayList<String>();

      al.add("Sachin");
      al.add("Rahul");


      // Typecasting is not needed
      String s1 = al.get(0);
      String s2 = al.get(1);
   }
}
```

4. Generics Promotes Code Reusability: With the help of generics in Java, we can write code that will work with different types of data. For example, public <T> void genericsMethod (T data) {...}

Here, we have created a generics method. This same method can be used to perform operations on integer data, string data, and so on.

5. Implementing Generic Algorithms: By using generics, we can implement algorithms that work on different types of objects, and at the same, they are type-safe too.

CONCLUSION :-

In a Internshala, this internship has been an excellent and rewarding experience. I can conclude that there have been a lot I've learnt from my work at Intershala. Needless to say, the technical aspects of the work I've done are not flawless and could be improved provided enough time. whatsoever I believe my time spent in research and discovering it was well worth it and contributed to finding an acceptable solution to build a fully functional web service. Two main things that I've learned the importance of are time-management skills, self-motivation and about Python, Sqlite etc….

PERUGU DEVANAND

`

PROJECT

SYSTECREATING A ONLINE BILLINGM IN JAVA

Problem:

Creating an entire online billing system involves a significant amount of code and different components. I can

provide you with a simplified example to get you started. However, keep in mind that a full-fledged online billing

system would involve databases, user authentication, security measures, and more. Here's a basic Java code

snippet to illustrate how you might structure the billing system:

Solution:

1. Create a Billing Entity:

Create a Java class for the billing entity to represent data:

```
import javax.persistence.Entity; import

javax.persistence.GeneratedValue; import

javax.persistence.GenerationType; import

javax.persistence.Id; import java.math.BigDecimal;

import java.time.LocalDate;

@Entity

public class Bill {

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

private Long id; private String

customerName; private

BigDecimal amount; private
```

```
LocalDate billDate;
// Constructors, getters, setters
}
```

2. Billing Repository:

Create a repository interface for the billing entity to perform database operations (assuming you're using Spring Data JPA):

```
import org.springframework.data.jpa.repository.JpaRepository; public
interface BillRepository extends JpaRepository<Bill, Long> {
// Custom queries if needed
}
```

3. Billing Service:

Create a service class to handle business logic for billing operations:

```
import org.springframework.beans.factory.annotation.Autowired; import
org.springframework.stereotype.Service;
import java.util.List;
@Service
public class BillingService {
private final BillRepository billRepository;
@Autowired
 public BillingService(BillRepository billRepository) { this.billRepository
= billRepository;
}
```

`

```
 public List<Bill> getAllBills() { return
billRepository.findAll();
}
return billRepository.save(bill);
}
// Other operations like update, delete, search by ID, etc.
}
```

4. Controller for REST APIs:

Create a controller to handle HTTP requests and map them to appropriate service methods:

```
import org.springframework.beans.factory.annotation.Autowired; import
org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/bills") public class
BillController { private final
BillingService billingService;
@Autowired
 public BillController(BillingService billingService) { this.billingService
= billingService;
}
@GetMapping
 public List<Bill> getAllBills() { return
billingService.getAllBills();
```

```
}

@PostMapping

public Bill createBill(@RequestBody Bill bill) {
return billingService.createBill(bill);

}

// Other endpoints for update, delete, search, etc.

}
```

5. Main Application Class:

Finally, create a main application class to run the Spring Boot application:

```
import org.springframework.boot.SpringApplication; import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication public class
BillingApplication { public static void
main(String[] args) {

SpringApplication.run(BillingApplication.class, args);

}

}
```

This provides a basic structure for an online billing system using Java and Spring Boot. It utilizes RESTful APIs to handle CRUD operations for billing data.

OUTPUT:

`


Billing System
Chrome 4
Copper 5
Plastic 3
Travel 4
Labour 4
Discount
0%
5%
10%
15%
20%
30%
Total Cost of Items £69.20
Total Cost of Labour £180.00
Total Cost of Mileage £20.00
Taxable £255.74
VAT £2.56
Tax £25.57
SubTotal £255.74
Total £283.87
Billing System:
Total
Reset
Receipt
Exit